pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Introduction to Python, Part 2

WEEK 1, DAY 1, PART 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's third note

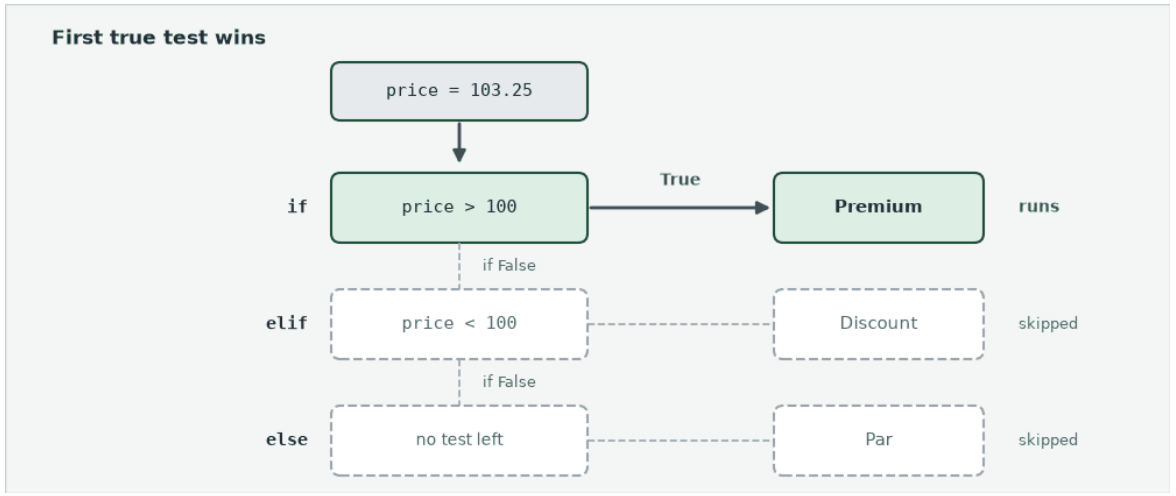
The book is in Python. Now make it work: tell me which of our four bonds are investment grade, cost every position in one go, and turn the coupon sum into something I can reuse.

Ticker	Issuer	Coupon (%)	Maturity	Price	Rating	Face held
QVNE	Quorvane Energy	5.25	2033	98.50	BBB	1,500,000
DNMR	Dunmarrow Rail	6.125	2034	103.25	BB+	500,000
PLWK	Pellwick Paper	4.75	2031	95.00	A-	750,000
HLVF	Halvering Foods	7.50	2032	101.00	B+	250,000

The desk holds 4 bonds and 3,000,000 face

1.6 Above, below, or at par

```
price = 103.25
if price > 100:
    print('DNMR trades at a premium')
elif price < 100:
    print('DNMR trades at a discount')
else:
    print('DNMR trades at par')
DNMR trades at a premium
```



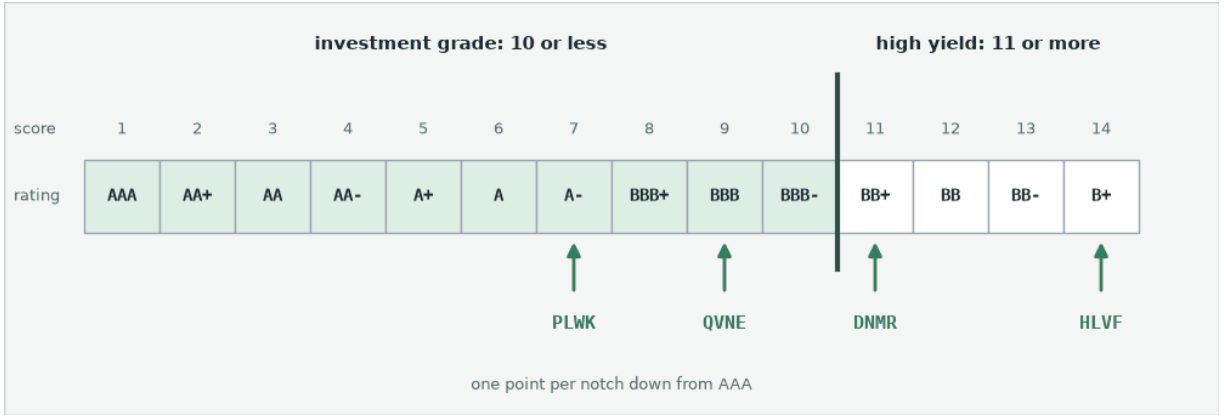
The tests are checked from the top: the first true one runs, the rest are skipped.

else catches what no test caught: here, exactly 100.

Excel side by side: a nested IF

```
=IF(A1>100,"Premium",IF(A1<100,"Discount","Par"))
```

Investment grade or high yield




```
rating_score = {'BBB': 9, 'BB+': 11, 'A-': 7, 'B+': 14}
score = rating_score['B+']
if score <= 10:
    print(f'HLVF scores {score}: investment grade')
else:
    print(f'HLVF scores {score}: high yield')
HLVF scores 14: high yield
```

A score of 10 (BBB-) or less is investment grade; HLVF's 14 is past the line, so the else block ran.

Compare and combine

Python	Asks	Excel
`==`	equal?	`=`
`!=`	not equal?	`<>`
`< and <=`	less? less or equal?	`< and <=`
`> and >=`	greater? greater or equal?	`> and >=`
`in and not in`	one of the items?	`=COUNTIF(A1:A4,"DNMR")>0`
`and`	both sides true?	`=AND(A1>10,B1>100)`
`or`	at least one side true?	`=OR(A1<=10,B1>100)`
`not`	flips True and False	`=NOT(A1<=10)`

==, two equals signs, tests; one = stores.

DNMR: rated BB+, priced at 103.25

```
score <= 10 and price > 100
False
score <= 10 or price > 100
True
not score <= 10
True
score == 10 or score == 11
DNMR sits on the investment grade line
```

Missing, empty, or false

```
second_rating = None
if second_rating is None:
    print('The second rating is missing')
else:
    print(f'The second rating is {second_rating}')
The second rating is missing
```

```
print(bool(0), bool(''), bool([]), bool(None))
print(bool(98.5), bool('QVNE'), bool([98.5]))
if face_list:
    print(f'The list holds {len(face_list)} positions')
False False False False
True True True
The list holds 4 positions
```

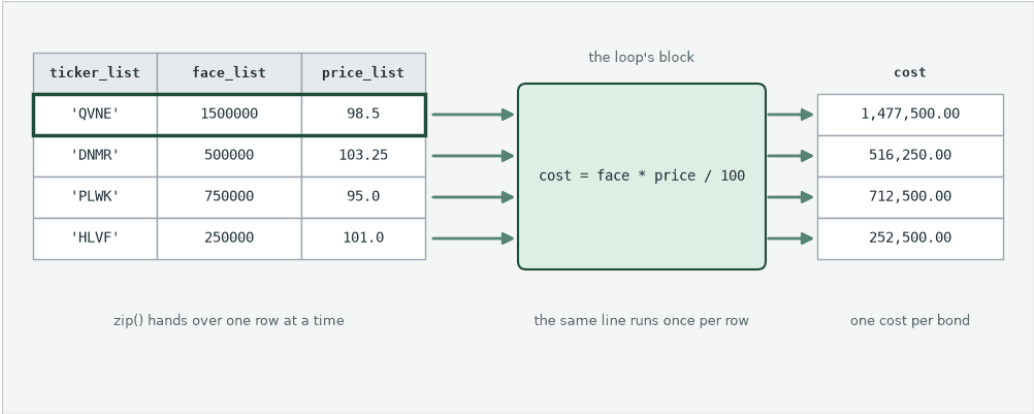
Excel side by side: an empty cell is the nearest thing, but not the same

Ask	Excel empty cell	Python None
Is it empty?	=ISBLANK(A1)	second_rating is None
Add 1	=A1+1 gives 1: blank counts as 0	None + 1 is a TypeError
A formula returning ""	ISBLANK says FALSE	'' is empty text, not None

Zero, empty text '', an empty list [], and None count as False; other numbers, text, and lists count as True.

1.7 One block, run once per item

```
face = 1500000
for test_price in range(96, 101):
    value = face * test_price / 100
    print(f'At {test_price} the position is worth ${value:,.2f}')
At 96 the position is worth $1,440,000.00
At 97 the position is worth $1,455,000.00
At 98 the position is worth $1,470,000.00
At 99 the position is worth $1,485,000.00
At 100 the position is worth $1,500,000.00
```




```
for ticker, face, price in zip(ticker_list, face_list, price_list):
    cost = face * price / 100
    print(f'{ticker} cost ${cost:,.2f}')
print(face, price)
QVNE cost $1,477,500.00
DNMR cost $516,250.00
PLWK cost $712,500.00
HLVF cost $252,500.00
250000 101.0
```

Excel side by side

Type =B2*C2/100 in one row and fill it down: the loop is the fill-down.

face is now 250000, not 1500000: a loop's name replaces an earlier one and keeps its last value.

Decide for every bond, total the book, loop to a target

Decide for every bond

```
for ticker, rating in zip(ticker_list, rating_list):
    if rating_score[rating] <= 10:
        print(f'{ticker} ({rating}) is investment grade')
    else:
        print(f'{ticker} ({rating}) is high yield')
QVNE (BBB) is investment grade
DNMR (BB+) is high yield
PLWK (A-) is investment grade
HLVF (B+) is high yield
```

Loop until a target is met

```
payment = 39375
collected = 0
payments = 0
while collected < 150000:
    collected += payment
    payments += 1
print(f'{payments} payments collect ${collected:,}')
4 payments collect $157,500
```

A running total

```
total_cost = 0
for face, price in zip(face_list, price_list):
    total_cost += face * price / 100
print(f'The book cost ${total_cost:,.2f}')
The book cost $2,958,750.00
```

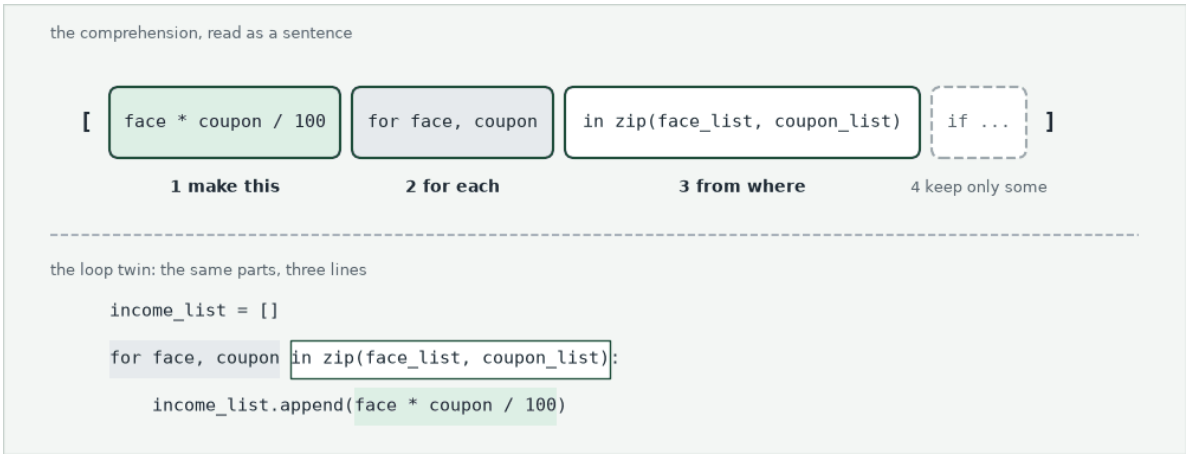
Excel side by side

Faces in B2:B5 and prices in C2:C5: =SUMPRODUCT(B2:B5,C2:C5)/100 gives it in one cell.

The note's first answer: QVNE and PLWK are investment grade.

The note's second answer: 2,958,750.00 dollars.

1.8 The same list, in one line

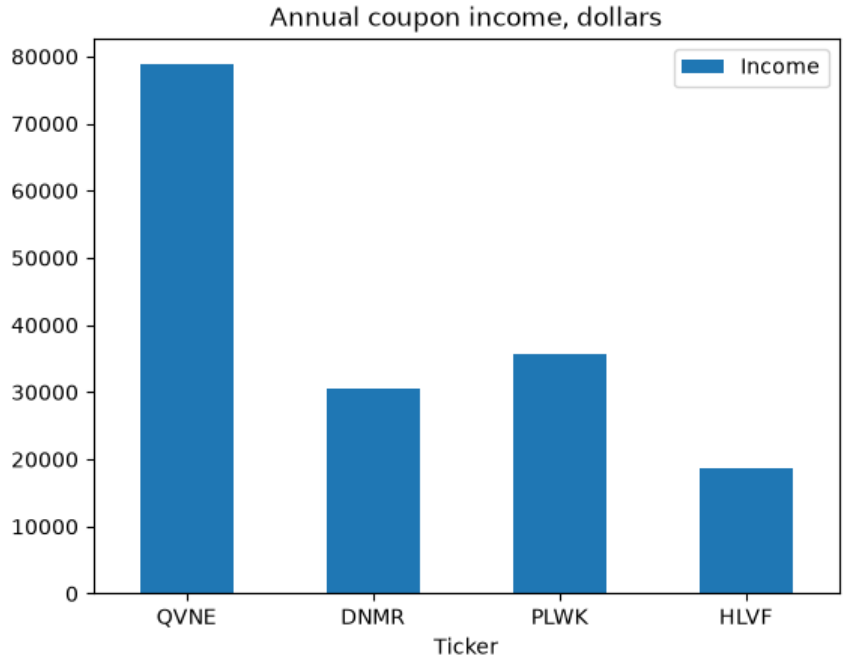


Excel side by side

Tickers in A2:A5 and scores in F2:F5: =FILTER(A2:A5,F2:F5<=10) keeps the same two; FILTER needs Excel 2021 or later, or Microsoft 365.

```
income_list = [face * coupon / 100 for face, coupon in zip(face_list, coupon_list)]
[78750.0, 30625.0, 35625.0, 18750.0]
label_list = ['Premium' if price > 100 else 'Discount' for price in price_list]
['Discount', 'Premium', 'Discount', 'Premium']
ig_tickers = [ticker for ticker, rating in zip(ticker_list, rating_list)
              if rating_score[rating] <= 10]
['QVNE', 'PLWK']
```

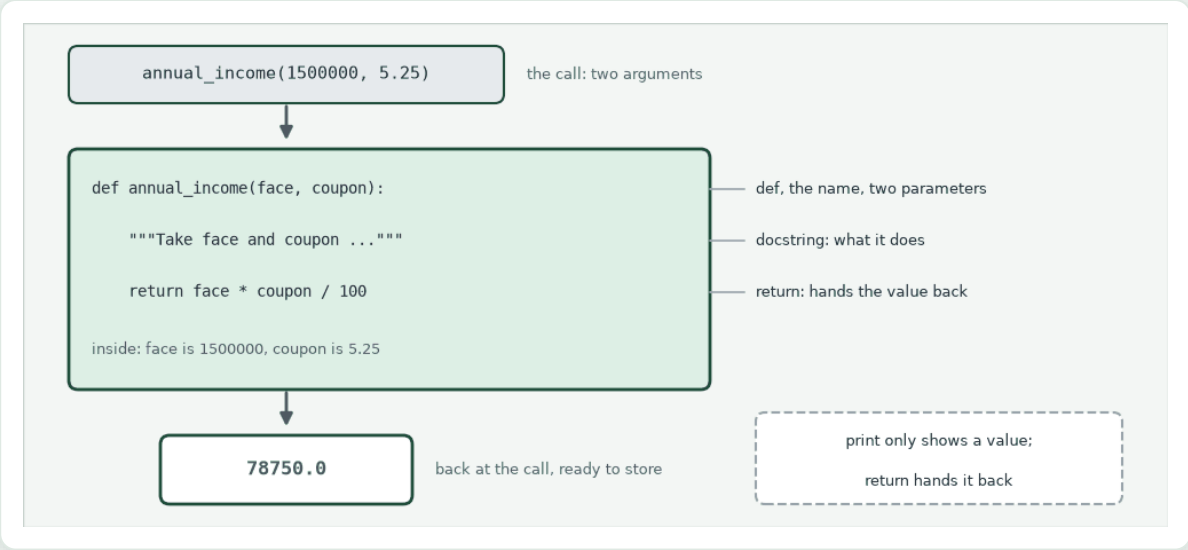
Draw the income



QVNE's bar towers over the rest: 78,750 of the book's 163,750 dollars a year, almost half.

```
table = pd.DataFrame({'Ticker': ticker_list, 'Income': income_list})
chart = table.plot.bar(x='Ticker', y='Income', rot=0, title='Annual coupon income, dollars')
```

1.9 Turn the coupon sum into a tool




```
def annual_income(face, coupon):
    return face * coupon / 100
qvne_income = annual_income(1500000, 5.25)
print(f'QVNE pays ${qvne_income:,.2f} a year')
QVNE pays $78,750.00 a year
```

Excel side by side

In Microsoft 365, LAMBDA does this job: in the Name Manager, name =LAMBDA(face,coupon,face*coupon/100) as ANNUAL_INCOME, and =ANNUAL_INCOME(1500000,5.25) gives 78750.

```
for ticker, coupon, price in zip(ticker_list, coupon_list, price_list):
    print(f'{ticker}: {current_yield(coupon, price):.2f} percent')
QVNE: 5.33 percent
DNMR: 5.93 percent
PLWK: 5.00 percent
HLVF: 7.43 percent
```

Defaults, sorting, and the book in one call

A default, and keywords

```
def total_income(incomes, months=12):
    return sum(incomes) * months / 12
A year: $163,750.00
Six months: $81,875.00
Three months: $40,937.50
```

Sort bonds by price

```
by_price = sorted(holdings, key=price_of, reverse=True)
print([holding['Ticker'] for holding in by_price])
['DNMR', 'HLVF', 'QVNE', 'PLWK']
```

Python	Excel
sorted(price_list)	=SORT(A1:A4)
sorted(price_list, reverse=True)	=SORT(A1:A4,1,-1)
sorted(holdings, key=price_of, reverse=True)	=SORTBY(A1:A4,C1:C4,-1)

The book in one call

```
book_face, book_cost, book_income = portfolio_summary(face_list, price_list, coupon_list)
Total face: 3,000,000   Total cost: $2,958,750.00   Annual income: $163,750.00
```

The note, answered

The note asked	Answer
Which bonds are investment grade	QVNE and PLWK; DNMR and HLVF are high yield
Cost every position in one go	one loop: 2,958,750.00 dollars
Make the coupon sum reusable	annual_income(face, coupon): 78,750.00 dollars a year for QVNE

Decide, repeat, package: most of the code you will ever write is built from these three, and you just used all of them on a book of bonds.

On Day 2 the book leaves the notebook: you write the holdings to a file in a folder of their own and read them back, with today's loops doing the walking.